

Assignment 1 : Sphere Neural Network

Renjiro Owen
Victoria University Wellington
Wellington, New Zealand
owenrenj@myvu.ac.nz

I. INTRODUCTION

Generative models learn to generate samples that resemble a desired probability distribution. In this assignment, fully connected neural networks are trained to learn transformations between distributions using explicit stochastic gradient descent (SGD) iterations.

First, a network f_1 maps a two-dimensional Gaussian distribution $Z \sim \mathcal{N}(0, I)$ to a two-dimensional uniform distribution over a unit square. A second network f_2 learns the inverse mapping from the uniform distribution back to a Gaussian distribution. The effect of L_1 and L_2 weight regularisation is also investigated by examining how regularisation changes the network weights and generated distribution.

Finally, a third network f_3 maps a one-dimensional uniform input to a two-dimensional Gaussian distribution. This demonstrates how a nonlinear neural network can learn a mapping from a lower-dimensional input to samples that approximate a higher-dimensional distribution.

II. THEORY

A generative model learns to produce samples that follow a desired probability distribution. A common approach is to learn a transformation from a simple source distribution, such as a Gaussian, to a target distribution using a neural network [1]. In this work, the neural network represents a function f_θ whose parameters θ are learned from samples of the source and target distributions.

The networks are trained using stochastic gradient descent (SGD). At each iteration, a minibatch of samples is generated, a loss measuring the difference between the generated and desired distributions is calculated, and the network parameters are updated according to

$$\theta \leftarrow \theta - \eta \nabla_\theta L, \quad (1)$$

where η is the learning rate and L is the training loss. SGD is widely used for training neural networks because it provides an efficient approximation of the gradient using batches of samples [1].

Regularisation can be used to control the size of neural network weights and reduce model complexity. In this work, both L_1 and L_2 regularisation are considered. L_1 regularisation penalises the absolute values of the weights, while L_2 regularisation penalises their squared values [1].

The L_1 regularisation term is defined as

$$\Omega_{L_1}(w) = |w|_1 = \sum_i |w_i|.$$

The L_2 regularisation term is defined as

$$\Omega_{L_2}(w) = \frac{1}{2} |w|_2^2 = \frac{1}{2} \sum_i w_i^2.$$

The regularised loss used for training can therefore be written as

$$L_{\text{MMD}} + \lambda_1 \Omega_{L_1}(w) + \lambda_2 \Omega_{L_2}(w),$$

where L_{MMD} measures the difference between the generated and target distributions, while λ_1 and λ_2 control the strength of the L_1 and L_2 penalties, respectively. L_1 regularisation can encourage sparse weights, whereas L_2 regularisation encourages weights to remain small [1].

III. EXPERIMENTS

A. Experimental Setup

1) *F1*: The f_1 generator uses a fully connected neural network with two hidden layers of 64 neurons and ReLU activations, followed by a Sigmoid output layer. It maps $z \sim \mathcal{N}(0, I)$ to two-dimensional samples in $[0, 1]^2$.

Training used a batch size of 256, learning rate of (0.01), and 5,000 iterations. The MMD loss with a Gaussian kernel $\sigma = 0.1$ was used to measure the difference between generated samples and samples from $U([0, 1]^2)$. After calculating the MMD loss, gradients were obtained through backpropagation and each network parameter was updated according to

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{MMD}},$$

The MMD loss was recorded at every iteration to monitor the training process.

2) *F2*: The f_2 generator uses a fully connected neural network with three hidden layers of 64 neurons using Tanh activations, followed by a two-dimensional linear output layer. The model maps $z \sim U([0, 1]^2)$ to the target distribution $\mathcal{N}(0, I)$.

Training used a batch size of 512, learning rate of (0.01), and 10,000 iterations. The MMD loss with a Gaussian kernel ($\sigma = 0.1$) was used to compare the generated and target distributions. Parameters were updated using explicit SGD.

3) *L1 L2*: The f_1 architecture was extended with L1 and L2 regularisation to investigate their effects on generator training. The batch size, learning rate, and number of iterations were kept at 256, 0.01, and 5,000, respectively. The total loss was defined as the MMD loss plus weighted L1 and L2 penalties.

Experiments were conducted using $\lambda = 0.001, 0.005$, and 0.01 individually for L1 and L2 regularisation. Additional experiments applied both L1 and L2 simultaneously using $\lambda = 0.001$ and 0.01 . This allowed the individual and combined effects of the regularisation terms on the generated distribution to be compared.

4) *f3*: The f_3 generator uses a one-dimensional input $y \sim U([0, 1])$, which is transformed into nonlinear Fourier features using sine and cosine functions. The number of features was varied between 4, 8, and 16 to investigate the effect of feature complexity on the generated distribution. These features are passed through two fully connected hidden layers of 64 neurons with Tanh activations, followed by a two-dimensional linear output layer. The model maps the one-dimensional uniform input to the target distribution $\mathcal{N}(0, I)$.

For each feature configuration, training used a batch size of 512, learning rate of 0.01, and 5,000 iterations. The MMD loss with a Gaussian kernel ($\sigma = 0.1$) was used to compare the generated samples with samples from the target distribution. Parameters were updated using explicit SGD.

B. Test Performance

1) *f1*: The f_1 generator was trained for 5,000 iterations using a learning rate of 0.01 and a batch size of 256. The MMD loss decreased substantially from 0.851574 at the start of training to 0.005059 at iteration 4,500. Although some fluctuations occurred during training, the overall trend shows that the generator successfully learned the target distribution.

The generated samples were then compared visually with the target distribution. As shown in Figure 1, the generated distribution is very similar to the target $U([0, 1]^2)$, with only small random variations. This indicates that f_1 was able to effectively approximate the target distribution.

2) *f2*: The f_2 generator was trained for 10,000 iterations using a learning rate of 0.01 and a batch size of 512. The MMD loss decreased substantially from 0.931694 at the start of training to approximately 0.005 during the later stages. Although the loss fluctuated slightly throughout training, it remained relatively low, indicating that the generated distribution became closer to the target distribution.

However, as shown in Figure 2, the generated samples form a diamond-shaped distribution concentrated around the centre of the target Gaussian distribution. While the generator successfully learned the approximate central location of the target, it did not reproduce the Gaussian shape and spread accurately. This suggests that a low MMD loss does not necessarily mean that the generated distribution visually matches the target distribution.

3) *L1 and L2 Regularisation*: The effect of increasing L1 regularisation can be observed by comparing Figures 3, 4, and 5. As the L1 coefficient increases from 0.001 to 0.01,

the distributions on the left and bottom right are progressively pushed towards the centre. This causes these regions to become slightly denser, while reducing the spread of the generated samples.

The effect of L2 regularisation is shown in Figures 6, 7, and 8. Increasing the L2 coefficient mainly reduces the overall range of the generated distribution. The samples become more concentrated towards the centre, resulting in a smaller overall spread.

When L1 and L2 are applied together, as shown in Figures 9 and 10, the effects of both regularisation terms become more apparent. The distribution shrinks towards the centre while the left and top regions become denser. Overall, stronger combined regularisation produces a more concentrated distribution with reduced spread and increased density in these regions.

4) *f3*: The effect of increasing the number of nonlinear features can be seen in Figures 11, 13, and 15. With 4 features, the generated samples form a V-shaped curve. Increasing the features to 8 produces a more curved distribution. With 16 features, the generated samples become more complex, circling around the centre multiple times. This shows that increasing the number of nonlinear features gives the generator greater flexibility to produce more complex patterns.

The loss results also show that the MMD loss generally decreases as more features are added. This indicates that increasing the number of features improves the generator’s ability to match the target distribution according to the MMD metric, although the generated shape does not necessarily become more visually similar to the target Gaussian.

IV. CONCLUSION

This study investigated neural network generators for learning target distributions using MMD. The f_1 generator successfully learned the uniform distribution, producing samples very similar to the target. In contrast, f_2 achieved a low MMD loss but produced a diamond-shaped distribution rather than accurately reproducing the Gaussian shape, showing that a low MMD does not always indicate a good visual match.

The f_3 experiments showed that increasing the number of nonlinear features allowed the generator to produce increasingly complex patterns, while generally reducing the MMD loss. L1 and L2 regularisation also affected the generated distributions by increasing concentration and reducing their overall range. Overall, the results demonstrate that architecture, feature complexity, and regularisation have an important effect on the quality of generated distributions.

V. STATEMENT

Tools Used

- Python 3
- Google Colab
- PyTorch
- Matplotlib
- ChatGPT [2] (used for proofreading and improving the report wording)

I declare that this assignment is my own work. The implementation, experimentation, model training, and evaluation were completed by me. Artificial intelligence tools were used only for language assistance during the preparation of this report.

Google Colab Code

Google Drive link

REFERENCES

- [1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [2] OpenAI, “ChatGPT (gpt-5.5),” <https://chatgpt.com/>, 2026, large language model. Accessed: 2026-07-20.

APPENDIX

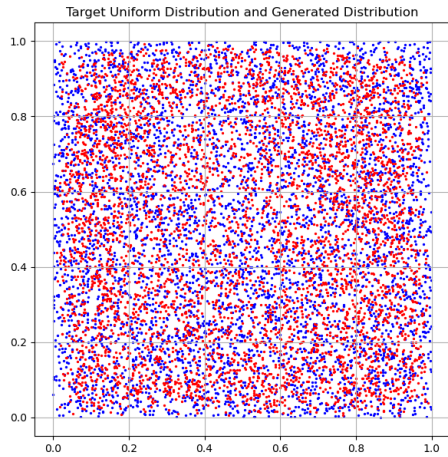


Fig. 1. Comparison between the target distribution and the generated distribution for f_1 .

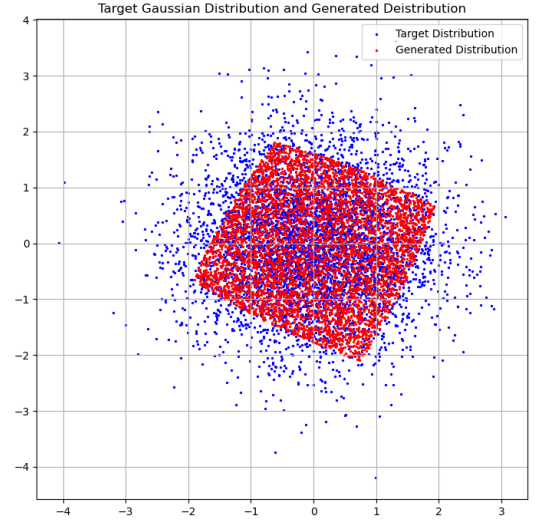


Fig. 2. Comparison between the target distribution and the generated distribution for f_2 .

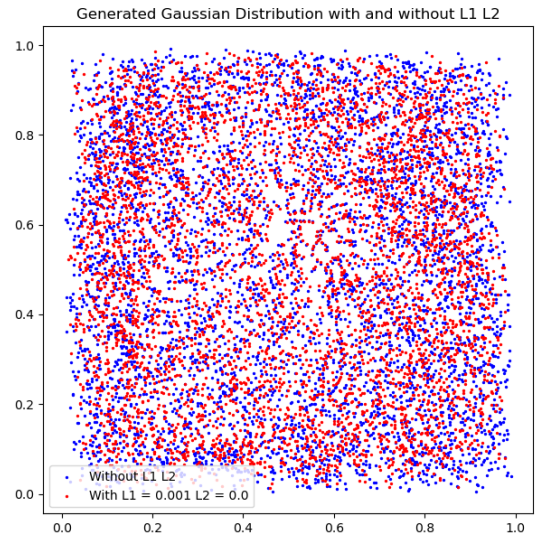


Fig. 3. Generated distribution for f_1 using L1 regularisation with $\lambda_{L1} = 0.001$.

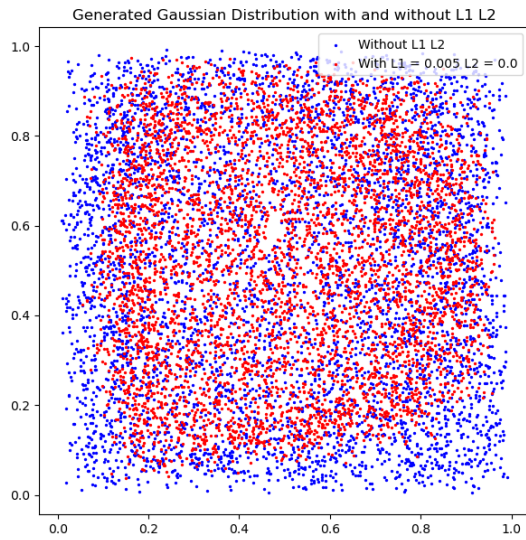


Fig. 4. Generated distribution for f_1 using L1 regularisation with $\lambda_{L1} = 0.005$.

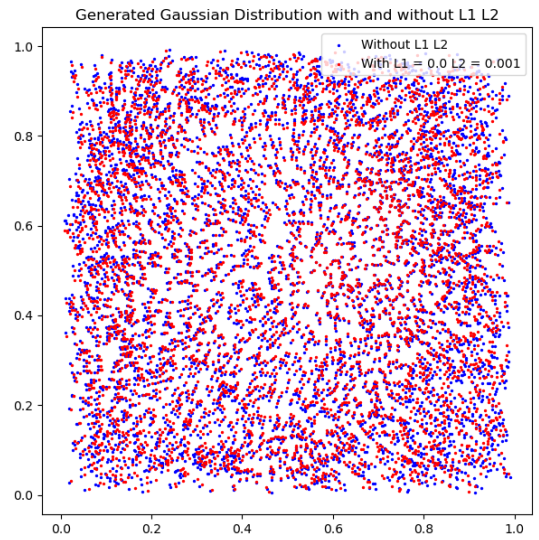


Fig. 6. Generated distribution for f_1 using L2 regularisation with $\lambda_{L2} = 0.001$.

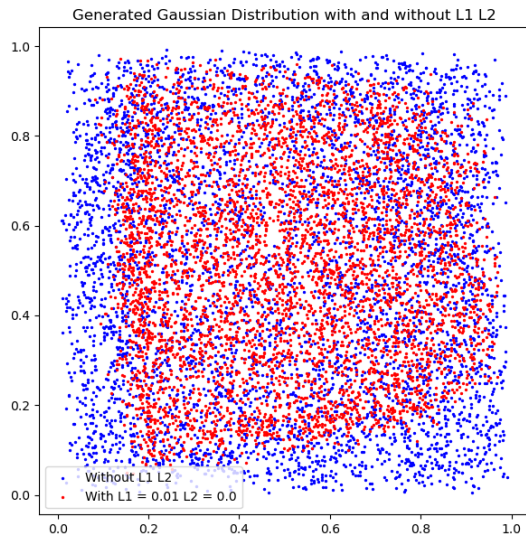


Fig. 5. Generated distribution for f_1 using L1 regularisation with $\lambda_{L1} = 0.01$.

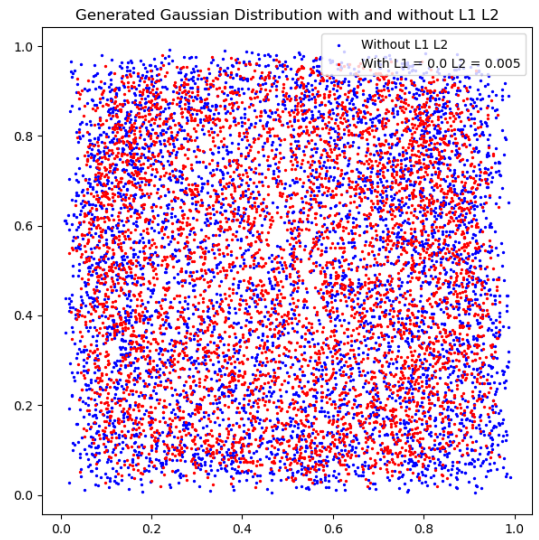


Fig. 7. Generated distribution for f_1 using L2 regularisation with $\lambda_{L2} = 0.005$.

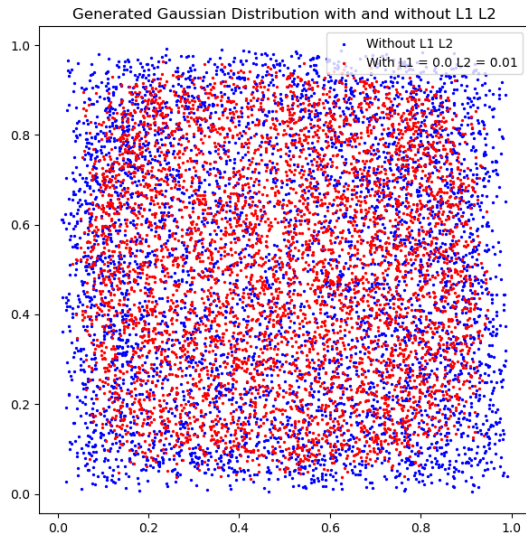


Fig. 8. Generated distribution for f_1 using L2 regularisation with $\lambda_{L2} = 0.01$.

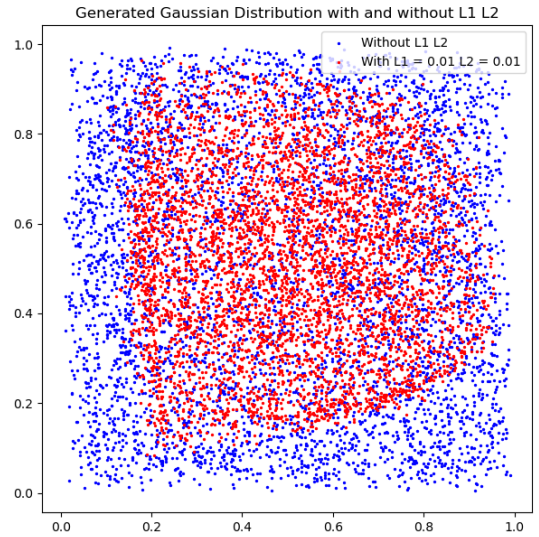


Fig. 10. Generated distribution for f_1 using both L1 and L2 regularisation with $\lambda_{L1} = \lambda_{L2} = 0.01$.

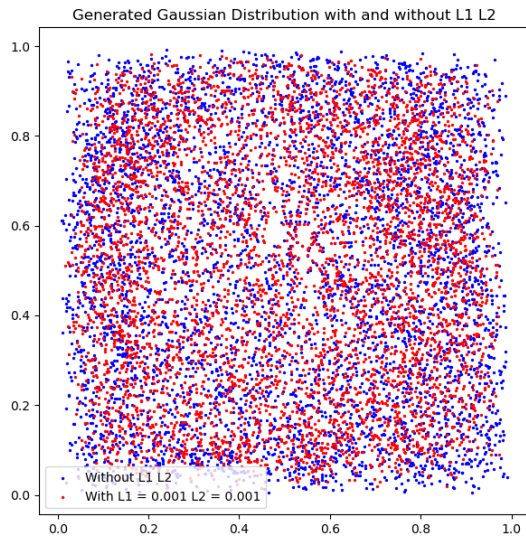


Fig. 9. Generated distribution for f_1 using both L1 and L2 regularisation with $\lambda_{L1} = \lambda_{L2} = 0.001$.

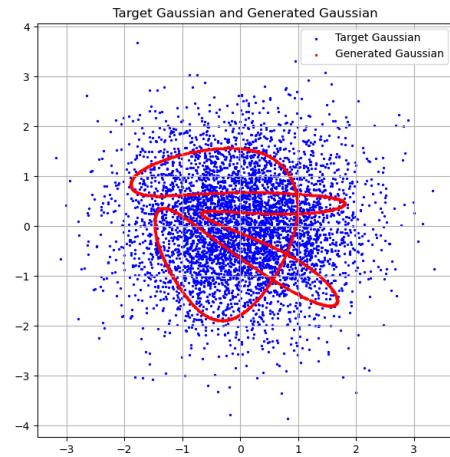


Fig. 11. Generated distribution for f_3 using 4 nonlinear features compared with the target distribution.

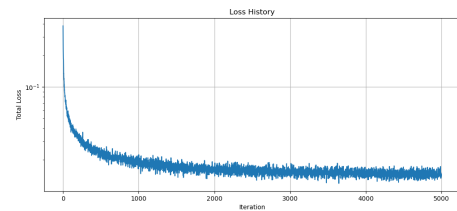


Fig. 12. Training loss for the generator with 4 nonlinear features.

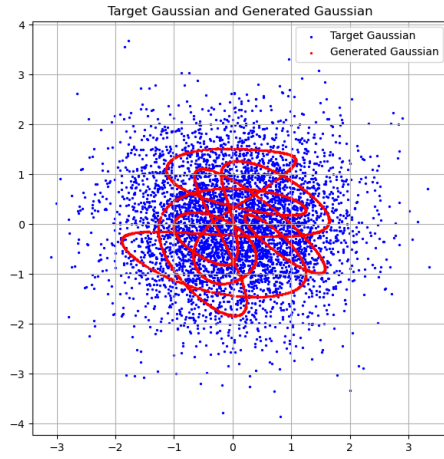


Fig. 13. Generated distribution for f_3 using 8 nonlinear features compared with the target distribution.

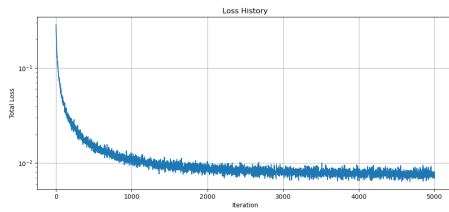


Fig. 14. Training loss for the generator with 8 nonlinear features.

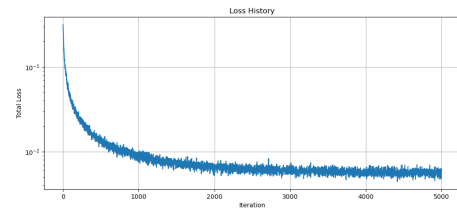


Fig. 16. Training loss for the generator with 16 nonlinear features.

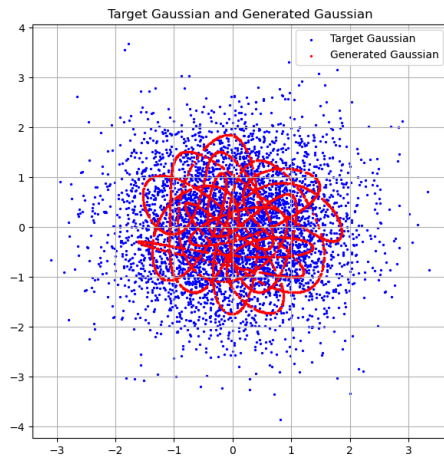


Fig. 15. Generated distribution for f_3 using 16 nonlinear features compared with the target distribution.